

Activity 3 - Share inventory records with ownership and borrowing

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K1/A1 K3/A3

Specification

Item { sku, name, qty: u32 }. total_quantity(&[Item]) -> u64 reads; restock(&mut Item, u32) -> Option<u32> changes one item and leaves it unchanged on overflow; in_stock(Vec<Item>) -> Vec<Item> consumes the list.

Part 1: Learn Rust prompts

Explain it simply

I am learning Rust. Explain ownership, moves, Copy types, clone, shared borrows (&) and mutable borrows (&mut) to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

Walk me through the sample

Here is samples/01_move_and_clone.rs from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

Quiz me

Quiz me on ownership, moves, Copy types, clone, shared borrows (&) and mutable borrows (&mut) with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles ownership and borrowing (memory management) with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

Part 2: Build with AI (vibe-coding prompts)

Plan before code

Act as a senior Rust reviewer. Specification: Item { sku, name, qty: u32 }. total_quantity(&[Item]) -> u64 reads; restock(&mut Item, u32) -> Option<u32> changes one item and leaves it unchanged on overflow; in_stock(Vec<Item>) -> Vec<Item> consumes the list. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Share inventory records with ownership and borrowing" in starter/src so that the supplied tests pass. Choose &, &mut or by-value parameters to match each function's intent; avoid clone() unless you justify it. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

Part 3: Rust sample scripts (samples/ folder)

01_move_and_clone.rs

Run: rustc --edition 2021 01_move_and_clone.rs && ./01_move_and_clone

```
// Sample: moving ownership, cloning, and Copy types.
fn consume(items: Vec<String>) -> usize {
    items.len() // `items` is dropped at the end of this function
}

fn main() {
    let items = vec![String::from("cable"), String::from("charger")];
    let copy = items.clone(); // explicit deep copy
    let count = consume(items); // ownership moves into `consume`
                                // println!("{:?}", items); // error[E0382]: borrow of moved value
    println!("consumed {count} items; the clone still has {copy:?}");
    let n: u32 = 5;
    let m = n; // integers are Copy: both stay usable
    println!("n = {n}, m = {m}");
}
```

Expected output:

```
consumed 2 items; the clone still has ["cable", "charger"]
n = 5, m = 5
```

02_borrowing.rs

Run: rustc --edition 2021 02_borrowing.rs && ./02_borrowing

```
// Sample: shared borrows read, a mutable borrow changes one element.
fn total(stock: &[u32]) -> u32 {
    stock.iter().sum()
}

fn restock(level: &mut u32, amount: u32) {
    *level += amount;
}

fn main() {
    let mut stock = vec![5, 0, 7];
    println!("total before = {}", total(&stock)); // shared borrow
    restock(&mut stock[1], 10); // exclusive borrow of one element
    println!("stock after  = {stock:?}");
    println!("total after  = {}", total(&stock));
}
```

Expected output:

```
total before = 12
stock after  = [5, 10, 7]
total after  = 22
```

Learn more

Rust Book: What Is Ownership?: <https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html>

Rust Book: References and Borrowing:

<https://doc.rust-lang.org/book/ch04-02-references-and-borrowing.html>

W3Schools: Rust Ownership: https://www.w3schools.com/rust/rust_ownership.php

Programiz: Rust Ownership: <https://www.programiz.com/rust/ownership>

Programiz: References and Borrowing: <https://www.programiz.com/rust/references-and-borrowing>

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>

W3Schools: Get Started with Rust: https://www.w3schools.com/rust/rust_getstarted.php